StarForth VM and Physics-Driven Adaptive Runtime

Volume I of the StarshipOS Technical Reference

Robert A. James

Contents

Chapter 1

Introduction

1.1 What StarForth Is

1.2 Technology Stack

Table 1.1: StarshipOS technology stack

Layer	Component
Future OS	StarshipOS (self-hosting)
Kernel	LithosAnanke v1.0.8 / StarKernel (bare-metal UEFI)
VM	StarForth v3.0.3 (FORTH-79, physics-driven adaptive RT)
Host platform	Linux, L4Re/Fiasco.OC, bare metal (amd64, aarch64, riscv64)

1.3 Thermodynamic Modeling Language

1.4 Document Scope and Organization

1.5 Experimental Campaign Notes

1.6 StarForth, LithosAnanke, and StarshipOS: A Technical Overview

Author: Robert A. James. Date: 9 April 2026. Status: Working Document.

1.6.1 StarForth

What It Is

StarForth is a pure C99 FORTH-79 virtual machine with no libc dependencies. It is self-adaptive: it observes its own execution behavior and continuously adjusts internal parameters to maintain a conservation invariant called K≡1.0. It runs on x86_64, aarch64, and RISC-V.

Why FORTH

FORTH is correct for this work for reasons beyond familiarity. FORTH words are discrete, classifiable, observable computational units — each word has a definite stack-effect signature, each word consumes a known quantity of computational energy, each word is a particle with measurable properties. This is the foundation of StarForth's self-adaptive behavior.

The Primitive Taxonomy

StarForth implements 23 word modules (18 FORTH-79 standard, 5 StarForth extensions). Every primitive word carries quantum-analog properties derived from its stack-effect signature:

Spin. Derived from net stack-effect arithmetic. A word consuming two values and producing one has spin $-\frac{1}{2}$; these values fall directly from `( n n - n )` notation, as half-integer spin falls from the Dirac equation.

Charge. The net stack delta. Charge conservation across a complete program means the program is stack-balanced.

Heat. Execution frequency and recency. Heat decays over time, analogous to radioactive decay.

Mass. Bytecode footprint of the word definition.

Color. Binding state: white words are fully resolved; colored words carry unresolved forward references.

Three of these quantities — spin, charge, and color — behave as genuine conservation laws across well-formed programs. This is a structural property of the system discovered empirically and subsequently formalized in Isabelle/HOL.

The SSM and James Law

The SSM (Steady State Machine) is the self-adaptive core of StarForth. It monitors execution behavior through instrumented subsystems and continuously adjusts internal parameters to maintain K≡1.0.

James Law ($K \equiv 1.0$) is a conservation invariant: the normalized total execution energy of the StarForth universe is conserved at unity across all configurations and workloads. Validation basis: 38,400 experimental runs spanning 128 factorial parameter configurations, CV below 1%, published on SSRN (abstract 5930134).

The SSM operates through four mechanisms:

Rolling Window of Truth (RWOT). A sliding observation window encoding five observables: word identity, prev→word transition, instruction pointer position, execution heat, and causal stack depth. Updated at heartbeat tick intervals, making it a coarse-grained lattice observable.

Adaptive Heartbeat. A self-adjusting timer governing when the SSM evaluates system state and adjusts parameters. The system's intrinsic rhythm.

Physics Hooks. Every word execution in the inner interpreter loop is bracketed by `physics_pre_execute` and `physics_post_execute` calls, updating heat, decay, and window measurements.

Decay. Execution heat decays between heartbeat ticks, keeping the observable window relevant to current rather than historical behavior.

The Phase Space and Attractor

Six workload types have been studied experimentally: STABLE, VOLATILE, TRANSITION, TEMPORAL, DIVERSE, and OMNI. Each produces a distinct trajectory in phase space. All trajectories converge to the same attractor: K≡1.0. Phase-space portrait visualizations confirm attractor convergence across all workload types on all three target architectures.

The Manifold Navigation Controller (Theoretical)

Given a bounded system with a known conservation invariant and predictable attractor manifold, a controller can:

1. Identify the system's current phase-space position using three observables: RWOT (position), adaptive heartbeat metadata (velocity), and spin-class transition rate (acceleration — requires instrumentation).
2. Compute a perturbance vector sufficient to transport the system from its current manifold to an arbitrary target manifold.
3. Inject the perturbance — the burn — and release.
4. Allow the system to coast back to K≡1.0 under its own restoring force.

The controller fires once and releases. The conservation invariant guarantees the return journey. This orbital-mechanics model scales from the word level through the VM, OS, FPGA, and prospectively to custom silicon.

—

1.6.2 LithosAnanke

What It Is

LithosAnanke is a bare-metal operating system for x86_64, currently at milestone M7 (VM integration). It boots via UEFI, manages physical and virtual memory, handles interrupts via IDT/APIC, maintains a timer, manages a heap, and hosts the StarForth VM as its primary computational substrate. Written in C99 with no external dependencies.

The Capsule System

The primary organizational unit is the *capsule*: a content-addressed, 64-byte cache-line-aligned descriptor. A capsule's ID is its content hash. Identity is content: two capsules with identical content have identical IDs. Modification produces a new capsule with a new ID. Capsules are immutable by design; mutation produces new capsules.

Component Architecture

Hera. The foundational VM and heartbeat. The system's ground state.

Hermes. Message entropy and TTL decay. Manages the message-passing fabric, monitors message lifecycle, and reaps expired messages. The natural home of the perturbance controller extension.

Artemis. Persistence and event sourcing. Manages the capsule store, maintains the event log, and handles durable state.

Components communicate exclusively through message passing. There is no shared mutable state between components.

Authentication and Identity

LithosAnanke implements a hardware-rooted authentication model with no usernames and no passwords. Identity is carried on a physical thumb drive containing two raw block partitions.

Access control is governed by temporal ACL grants that decay via the same Hermes TTL mechanism that governs message lifetime. The default state of the permission system is no access. Decay to zero requires no explicit revocation.

Normalized Virtual Block Space

From any VM's perspective, LithosAnanke presents a single contiguous block address space. Physical origin — local drive, system hardware, or remote cloud mount — is transparent. Blocks appear as grants are issued; blocks vanish as grants decay. Security is expressed as address-space geometry.

The Three UX Tiers

CLI. The traditional FORTH terminal interface. Full power, zero overhead. The REPL is operational.

GUI. A wireframe soundstage rendered from rasterized vector graphics. Words are physical objects; the user assembles colon definitions spatially. The FORTH text is always visible underneath.

VR. The full holodeck realization. The user stands inside the phase space. Words have mass, spin, heat, and charge as observable properties. Manifold navigation is literally navigable space.

All three tiers are windows into identical underlying mechanics.
—

1.6.3 StarshipOS

The SSPU

The SSPU (Steady State Processing Unit) is a novel processor architecture extending the StarForth/SSM work into hardware. It communicates entirely via a message-passing fabric called Gefyra. There are no traditional buses. Targeted for initial FPGA implementation on a Digilent Genesys ZU (Zynq UltraScale+).

Milestone Status

Milestone	Description	Status
M0	UEFI boot	Complete
M1	Physical memory manager	Complete
M2	Virtual memory manager	Complete
M3	IDT/APIC interrupt handling	Complete
M4	Timer	Complete
M5	Heap	Complete
M6	–	Complete
M7	VM integration	Active frontier
–	Manifold Navigation Controller	Theoretical
–	SSPU FPGA implementation	Planned (Genesys ZU)

Design Philosophy

The recurring pattern in this architecture is the elimination of problem domains rather than their simplification. No scheduler, no traditional memory manager, no username/password authentication, no logout ceremony, no coordination layer for distributed state — each elimination is a deliberate architectural decision. The question at each design point was not "how do we implement this?" but "do we need this at all?"

Key Terms

Term	Definition
K≡1.0	James Law — conservation invariant of normalized execution energy
SSM	Steady State Machine — self-adaptive core of StarForth
RWOT	Rolling Window of Truth — coarse-grained phase-space observable
SSPU	Steady State Processing Unit — novel message-passing processor
Gefyra	Message-passing fabric connecting SSPU processing elements
Capsule	Content-addressed, 64-byte aligned immutable descriptor
Hera	Foundational VM and heartbeat component
Hermes	Message entropy, TTL decay, lifecycle management
Artemis	Persistence and event sourcing

Chapter 2

FORTH-79 Interpreter

2.1 Memory Model

2.2 Dictionary Structure

2.3 FORTH-79 Word Categories

Table 2.1: StarForth word implementation files

File	Scope
`arithmetic_words.c`	`+ - * / MOD ABS MIN MAX`
`stack_words.c`	`DUP DROP SWAP ROT OVER NIP TUCK`
`control_words.c`	`IF ELSE THEN DO LOOP BEGIN UNTIL WHILE`
`defining_words.c`	`: ; CREATE DOES> VARIABLE CONSTANT`
`memory_words.c`	`@ ! C@ C! MOVE FILL`
`return_stack_words.c`	`>R R> R@ RDROP 2>R 2R@ 2R>`
`double_words.c`	`2DUP 2DROP 2SWAP 2@ 2! D+ D-`
`logical_words.c`	`AND OR XOR NOT INVERT LSHIFT RSHIFT`
`io_words.c`	`EMIT KEY TYPE CR TAB SPACE ACCEPT`
`string_words.c`	`S" SLITERAL` and string operations
`block_words.c`	`BLOCK BUFFER LOAD THRU FLUSH`
`format_words.c`	`.( .R .S HEX DECIMAL BASE`
`system_words.c`	`BYE ABORT INCLUDE STATE`
`dictionary_words.c`	`FIND SEARCH-WORDLIST WORDS`
`vocabulary_words.c`	`VOCABULARY DEFINITIONS FORTH-WORDLIST`
`q48_16_words.c`	$\mathbb{Q}_{48.16}$ fixed-point word definitions
`starforth_words.c`	StarForth-specific extensions

2.4 Initialization and Boot

2.5 Q48.16 Fixed-Point Arithmetic

Chapter 3

Physics-Driven Adaptive Runtime

3.1 Overview: Seven Feedback Loops

3.2 Loop #1 — Execution Heat

3.3 Loop #2 — Rolling Window of Truth

3.4 Loop #3 — Linear Decay

3.5 Loop #4 — Word Transition Prediction (Pipelining)

3.6 Loop #5 — Window Width Inference

3.7 Loop #6 — Decay Slope Inference

3.8 Loop #7 — Adaptive Heartrate

3.9 L8 Jacquard Mode Selector

3.10 Determinism and Formal Properties

3.11 Hot-Words Cache Optimization

3.12 Hot-Words Cache Performance Analysis

3.12.1 Summary

The StarForth hot-words cache achieves a statistically confirmed **1.78**× speedup for dictionary lookups, with a 95% Bayesian credible interval of $[1.75\times, 1.81\times]$. All measurements use Q48.16 fixed-point arithmetic; no floating-point computation is required.

3.12.2 Experimental Configuration

- **Build:** `make ENABLE_HOTWORDS_CACHE=1 fastest` (x86_64, full assembly optimisations, LTO, direct threading)
- **Benchmark:** 100,000 dictionary lookups via FORTH test harness; word mix: common FORTH primitives (`IF`, `DUP`, `DROP`, `+`, `-`, `@`, `!`, etc.)

- **Cache:** 32 entries; promotion threshold: 50 executions; eviction: LRU
- **Precision:** Q48.16 64-bit fixed-point (nanoseconds)

3.12.3 Lookup Distribution

Path	Count	Fraction
Cache hits	1,415	35.64%
Bucket hits	1,861	46.88%
Misses	694	17.48%
Total	3,970	100%

3.12.4 Latency Results

Path	Min (ns)	Mean (ns)	Max (ns)
Cache path (1,415 samples)	22	31.543	101
Bucket path (1,861 samples)	23	56.237	370

3.12.5 Speedup Analysis

$$\text{speedup} = \frac{\bar{t}_{\text{bucket}}}{\bar{t}_{\text{cache}}} = \frac{56.237\,\text{ns}}{31.543\,\text{ns}} = 1.78\times \tag{3.1}$$

Time saved per cache hit: $56.237 - 31.543 = 24.694$ ns (43.9% reduction).

Bayesian credible intervals.

Interval	Speedup
95% credible	$[1.75\times,\ 1.81\times]$
99% credible	$[1.73\times,\ 1.83\times]$

$P(\text{speedup} > 1.1\times) = 99.9\%$. All credible-interval arithmetic is performed in pure Q48.16 integer arithmetic; no floating-point or libm functions are used.

3.12.6 Cache Management Behaviour

The physics engine promoted 10 words automatically from the dictionary into the cache based on execution entropy exceeding the threshold of 50. No manual tuning was applied. Zero evictions occurred during the benchmark; the 32-entry cache was sufficient for the active working set.

The two highest-entropy words at experiment completion were `EXIT` (entropy 114) and `LIT` (entropy 101), consistent with their frequency in the compiled FORTH test harness.

3.12.7 Production Properties

- **No floating-point:** All arithmetic is Q48.16 integer; the cache subsystem is compatible with L4Re and bare-metal targets.
- **No external libraries:** No libm dependency.
- **Deterministic:** Near-zero variance across 3,970 samples confirms formally proven deterministic behaviour.
- **Linear complexity:** Cache lookup is O(1); bucket search is linear in bucket depth.

3.12.8 Test Suite Validation

```
make fastest ENABLE_HOTWORDS_CACHE=1
make test
# Total: 782  Passed: 731  Failed: 0  Skipped: 49  Errors: 0
```

All 731 implemented tests pass with the cache-enabled build.

3.13 Optimization Proposals

3.14 Physics-Driven Optimisation Proposals

3.14.1 Strategic Direction

The hot-words cache experiment (1.78×, 95% CI: [1.75×, 1.81×]) validates the physics-driven optimisation methodology: collect execution-frequency metrics at runtime, make automatic promotion decisions, and measure impact with statistical rigour. Nine additional opportunities build on the same foundation.

JIT compilation is explicitly excluded from consideration. JIT requires runtime code generation, violates L4Re microkernel policy, introduces floating-point overhead, and defeats formal verification. Physics-driven optimisation achieves measurable gains within StarForth's verification and portability constraints.

3.14.2 Opportunity Catalogue

Opportunity 1 — Return Stack Prediction and Colon Word Inlining. Frequently called colon definitions with small bodies (fewer than 256 bytes) are candidates for compile-time inlining when `execution_heat > 100`. Inlining eliminates dictionary lookup and return-stack overhead. Expected gain: 1.3–2.0× for call-heavy workloads. Verification: static (inlining correctness is provable via Isabelle/HOL).

Opportunity 2 — Block I/O Prefetching. A Markov-style transition matrix tracks which block LBN follows which. When a block is accessed and the next-block heat exceeds a threshold, that block is pre-fetched into the buffer cache. Expected gain: 1.5–3.0× for block-sequential access patterns. Implementation: medium complexity.

Opportunity 3 — Stack Operation Fusion. Co-execution heat tracks consecutive word pairs (e.g., `DUP DROP`, `SWAP ROT`, `OVER SWAP`). Pairs exceeding a heat threshold at compile time are fused into dedicated primitives, eliminating two lookups and one execution per pair. Expected gain: 1.2–1.5× for stack-heavy programs. Verification: static (composition of verified primitives).

Opportunity 4 — Memory Allocation Pattern Prediction. Allocation frequency by size is tracked in a heat vector. Sizes exceeding a threshold trigger pre-warmed pool maintenance. Repeating allocation sequences are detected and cached. Expected gain: 1.3–1.8× for allocation-heavy programs.

Opportunity 5 — Control Flow Branch Prediction. `IF`/`THEN`/`ELSE` outcomes are tracked per source location. Branches with greater than 90% one-sided bias are annotated with a prediction hint that the inner interpreter can use to reorder code or emit x86 branch-hint prefixes. Expected gain: 1.1–1.3× (architecture-dependent).

Opportunity 6 — Vocabulary Search Path Reordering. Per-vocabulary hit rates are tracked. The search order is sorted dynamically by hit rate, placing the most productive vocabulary first. Expected gain: 1.2–1.6× in multi-vocabulary programs. Implementation: low complexity.

Opportunity 7 — String Operation Batching. Consecutive string-output operations (`."`, `EMIT`, `TYPE`) detected at compile time are fused into a single batch output, reducing interpreter loop iterations. Expected gain: 1.1–1.4× for I/O-bound programs. Implementation: low complexity.

Opportunity 8 — Arithmetic Operation Reordering. For hot commutative operations at a given source location, operand sizes are compared and the smaller operand is loaded first to improve prefetch behaviour. Expected gain: 1.05–1.15× (architecture-dependent).

Opportunity 9 — Word Placement Optimisation. Co-execution heat between word pairs guides memory compaction: frequently co-executed words are relocated adjacent to each other in the dictionary to improve instruction-cache locality. Expected gain: 1.05–1.2×. Implementation: high complexity (requires GC integration).

3.14.3 Implementation Roadmap

Phase	Opportunities	Rationale
1 (complete)	Hot-words cache	Proven; production-ready
2 (recommended next)	#3, #6, #1	Highest ROI, lowest complexity
3 (future)	#2, #7, #4	Medium complexity
4 (advanced)	#5, #8, #9	CPU-specific or GC-dependent

3.14.4 Expected Cumulative Gains

Conservative sequential composition of Phases 1–2:

$$1.78\times \;\times 1.2 \;\times\; 1.2 \;\times\; 1.3 \;\approx\; 3.3\times \text{ cumulative speedup}$$

All nine opportunities together: 5–8× total improvement is plausible, subject to workload dependency and diminishing returns.

3.14.5 Unified Metrics Infrastructure

All nine opportunities require co-execution heat tracking and sequence pattern buffers not yet present in the VM. A one-time extension to the `PhysicsMetrics` per-word structure adds: a co-execution heat vector, timing accumulator, per-word cache-line counters, and a small recent-execution circular buffer. This single investment unlocks the infrastructure required for Opportunities 1–9.

Chapter 4

Formal Verification

4.1 Proof Infrastructure

4.2 Base Definitions

4.3 Physics Loop Proofs

4.3.1 Loop #1: Execution Heat

4.3.2 Loop #2: Rolling Window Bounded Growth

4.3.3 Loop #3: Decay Convergence

4.3.4 Loop #4: Pipelining Metrics Safety

4.3.5 Loop #5: Window Width Inference Termination

4.3.6 Loop #6: Decay Slope Inference Soundness

4.3.7 Loop #7: Heartrate Adaptive Stability

4.4 Word-Category Proofs

4.5 Concurrency and Mutex

4.6 Overall Correctness

4.7 ACL System Proofs

Chapter 5

Experimental Results

5.1 Experimental Design

5.2 Primary Result: Steady-State Convergence

5.3 Claim Table

5.4 Negative Results and Limitations

5.5 Sensitivity Analysis

5.6 Reproducing the Results

Chapter 6

Build and Deployment

6.1 Prerequisites

6.2 Build Targets

Listing 6.1: StarForth build targets

```
make                # Standard optimized build (auto-detects
    architecture)
make fastest        # Maximum performance (ASM + LTO + direct
    threading)
make pgo            # Profile-guided optimization
make debug          # Debug build with symbols
make test           # Full test suite (936+ tests)
make bench          # Quick benchmark
make clean          # Remove build artifacts
```

6.3 Build Flags

6.4 Running

Listing 6.2: Running StarForth

```
./build/amd64/standard/starforth              # Interactive REPL
./build/amd64/standard/starforth --run-tests  # Tests then REPL
./build/amd64/standard/starforth -c "1␣2␣+␣.␣BYE"  # Inline code
./build/amd64/fastest/starforth --doe         # DoE mode
```

6.5 Test Suite

Chapter 7

Word-Level ACL System

7.1 Design Summary

The word-level ACL system is implemented through Phase 6. Key constraints (cite the design doc for full details):

- All policy logic in `ACL.4th` — no new C primitives for policy.
- Four `DictEntry` C fields: `acl_ttl`, `acl_allow`, `acl_mode`, `acl_pinned`.
- Two VM flags: `emergency_console` (fault handler active) and `zuse_session` (superuser authenticated).
- `ACL.4th` is self-activating; `init.4th` only needs `S" ACL.4th" EXEC`.
- `ACL-PIN` is one-way; inheritance clears pin, copies mode.

7.2 Phase Status

Table 7.1: ACL implementation phase status

Phase	Description	Status
1	C Infrastructure (`DictEntry` fields, interpreter hook, `acl_recheck()`)	Done
2	`ACL.4th` FORTH policy words + `ACL-INIT-PRIMITIVES` + self-activation	Done
3	`capsules/zuse.4th` bootstrap superuser skeleton; CA root placeholder	Done
4	`init.4th` opt-in toggle	Done
5	POST tests (800/800) + Isabelle/HOL proofs (5 theory files)	Done
6	`EMERGENCY_CONSOLE_ENABLED` build flag + `vm_fault_handler`	Done
7	LithosAnanke parity — port to kernel context, three-arch acceptance	Pending
8	PKI / thumbdrive — Ed25519 challenge-response; user minting by zuse	Future

7.3 ACL Formal Proofs

Five Isabelle/HOL theorems cover the ACL system's core safety properties:

1. **Pin Monotonicity** (`ACL_Pin_Monotone.thy`) — once pinned, a word remains pinned through the system's lifetime.

2. **Inheritance Clears Pin** (`ACL_Inherit_Clears_Pin.thy`) — inherited words carry mode but never carry pin status.

3. **TTL Bounded** (`ACL_TTL_Bounded.thy`) — all TTL values remain within defined bounds; no overflow.

4. **Emergency Bypass Correct** (`ACL_Emergency_Bypass.thy`) — the emergency console bypass does not escalate privileges.

5. **No Escalation** (`ACL_No_Escalation.thy`) — no sequence of ACL operations allows a word to acquire permissions it was not granted.